Project Model, API, Angular & SQL Integration Guide

AI-Driven Weather Forecasting and Crop Recommendation System for Anuradhapura District

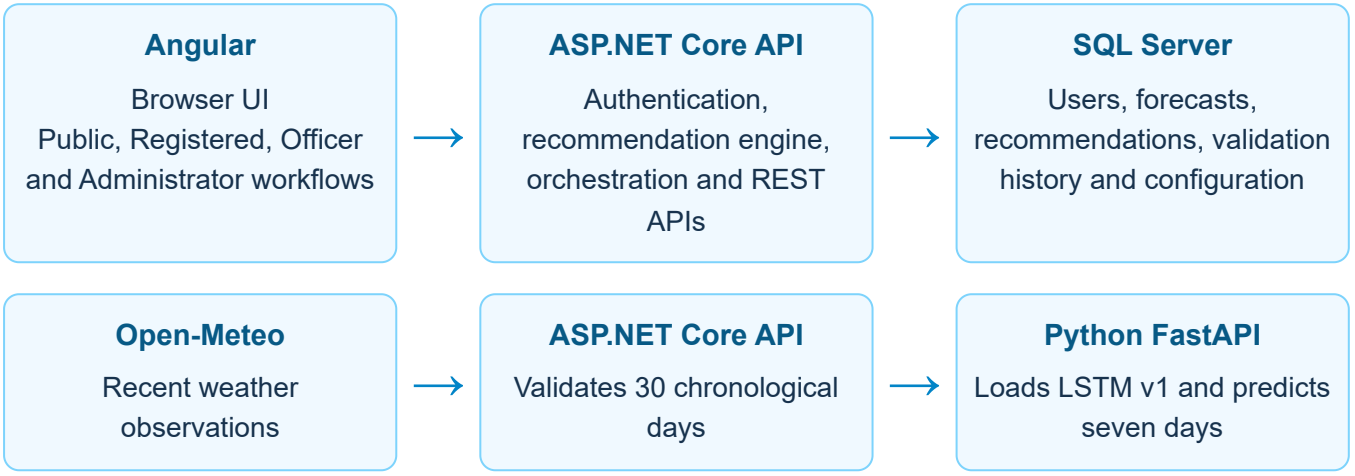
Angular | ASP.NET Core | Python FastAPI | LSTM v1 | SQL Server

1. What the system contains

The system is deliberately split into three runnable projects. Each project has one responsibility, so the Angular user interface does not load or train the model directly.

Project	Folder	Responsibility
Angular frontend	`src/frontend`	Displays forecasts, recommendations, history, Officer review and Administrator pages.
ASP.NET Core backend	`src/backend/AnuradhapuraAI.Api`	Provides secure APIs, authentication, suitability logic, SQL persistence and orchestration.
Python forecasting service	`src/forecasting-service`	Loads the saved LSTM v1 model and returns a seven-day weather forecast.

2. Overall architecture



Important: the Python service predicts weather only. Crop suitability remains in ASP.NET Core, where the approved six-crop explainable calculation is applied.

3. LSTM v1 to SQL to Angular workflow

1. An Administrator requests an operational forecast.
2. ASP.NET Core obtains the approved recent weather observations from Open-Meteo.
3. The backend validates that exactly 30 daily observations are chronological and complete.
4. The backend sends those observations to the Python `POST /forecast` endpoint.
5. The Python service loads the saved `v1` model and scaler, then returns exactly seven rainfall, temperature and humidity predictions.
6. The backend validates the returned model version, dates and ranges, then stores a `ForecastRun` and seven `ForecastRecord` rows in SQL Server.
7. Angular calls `GET /api/forecasts/latest` and displays the stored forecast.
8. Angular calls the recommendation API; ASP.NET Core evaluates the approved crop rules against the stored seven-day forecast and returns factor-level explanations.

How to Run the Three Projects

Use three terminals. Keep each terminal open while testing.

4. Before you start

- Use the project root: `D:\My\MSc\Assignment\Final Project\Reports\app`.
- Start the backend before Angular.
- Start the Python service before using Administrator operational forecast generation.
- Do not run LSTM training commands. The commands below only load the existing trained model.

5. Run ASP.NET Core API — Visual Studio or terminal

Option A: Visual Studio

1. Open `src/backend/AnuradhapuraAI.slnx` in Visual Studio.
2. Set `AnuradhapuraAI.Api` as the startup project.
3. Select the **https** launch profile.
4. Press **F5** or click the green Run button.
5. Confirm the Output window shows the API listening at `https://localhost:7055`.

Option B: VS Code or terminal

```
cd "D:\My\MSc\Assignment\Final Project\Reports\app"
dotnet run --project .\src\backend\AnuradhapuraAI.Api\AnuradhapuraAI.Api.csproj --launch-profile
https
```

Verify the backend in a browser:

```
https://localhost:7055/health
```

> If the browser reports an untrusted local certificate, run `dotnet dev-certs https --trust` once, then restart the API.

6. Configure Angular to use the HTTPS API

Angular keeps API calls as `/api/...`. Its local proxy forwards them to the ASP.NET Core API. In `src/frontend/proxy.conf.json`, use:

```
{
  "/api": {
    "target": "https://localhost:7055",
    "secure": false,
    "changeOrigin": true
  }
}
```

`secure: false` is required only for the locally generated HTTPS development certificate. Restart Angular after saving this file.

7. Run Angular — VS Code terminal

```
cd "D:\My\MSc\Assignment\Final Project\Reports\app\src\frontend"
npm install
npm start
```

Open `http://localhost:4200`. The frontend proxies its API requests securely to `https://localhost:7055/api/...`.

Run the Trained LSTM Model

8. Run Python FastAPI — VS Code terminal

```
cd "D:\My\MSc\Assignment\Final Project\Reports\app\src\forecasting-service"
python -m uvicorn forecasting_service.api:app --host 127.0.0.1 --port 8000
```

Verify that the trained model is loaded:

```
http://127.0.0.1:8000/health
```

The expected response contains `"status": "Healthy"` and `"modelVersion": "v1"`.

> **No virtual environment?** If the command reports a missing package, create one once: `python -m venv .venv`, activate it with `.\.venv\Scripts\Activate.ps1`, then run `python -m pip install -e .`.

9. Startup order and manual test checklist

Order	Service	Address / check	Why it is needed
1	ASP.NET Core API	`https://localhost:7055/health-ui`	Required for Angular, SQL access, authentication and recommendations.
2	Python LSTM service	`http://127.0.0.1:8000/health-ui`	Required only when generating a fresh operational forecast.
3	Angular	`http://localhost:4200`	Browser application for manual testing.

Each `/health-ui` address is a browser-friendly, read-only dashboard. The existing `/health` addresses remain the machine-readable JSON health endpoints.

1. Open the dashboard and confirm a stored seven-day forecast appears.
2. Open Crop Recommendation and generate a public evaluation.
3. Register or log in as a Registered User; generate a recommendation and check History.
4. Log in as the authorized Agricultural Officer and review the safe QA recommendation.
5. Log in as Administrator. With the Python service running, use Generate operational forecast only when you intend to create the approved local-development operational run.

10. Quick troubleshooting

Message / symptom	Check	Safe correction

“The request could not be completed.”	Angular proxy and API use different ports.	Ensure API is on `https://localhost:7055`, update the proxy as shown, then restart Angular.
Recommendation or login fails	Open API Output/terminal for the HTTP status and safe message.	Confirm the API health endpoint and local database connection. Do not edit production configuration.
Operational forecast says service unavailable	Python service may not be running on port 8000.	Start FastAPI and check `/health`.
Certificate warning	.NET development certificate is not trusted.	Run `dotnet dev-certs https --trust` once.

This guide intentionally omits database server addresses, usernames, passwords and connection strings. Keep secrets in local user-secrets and never commit them.